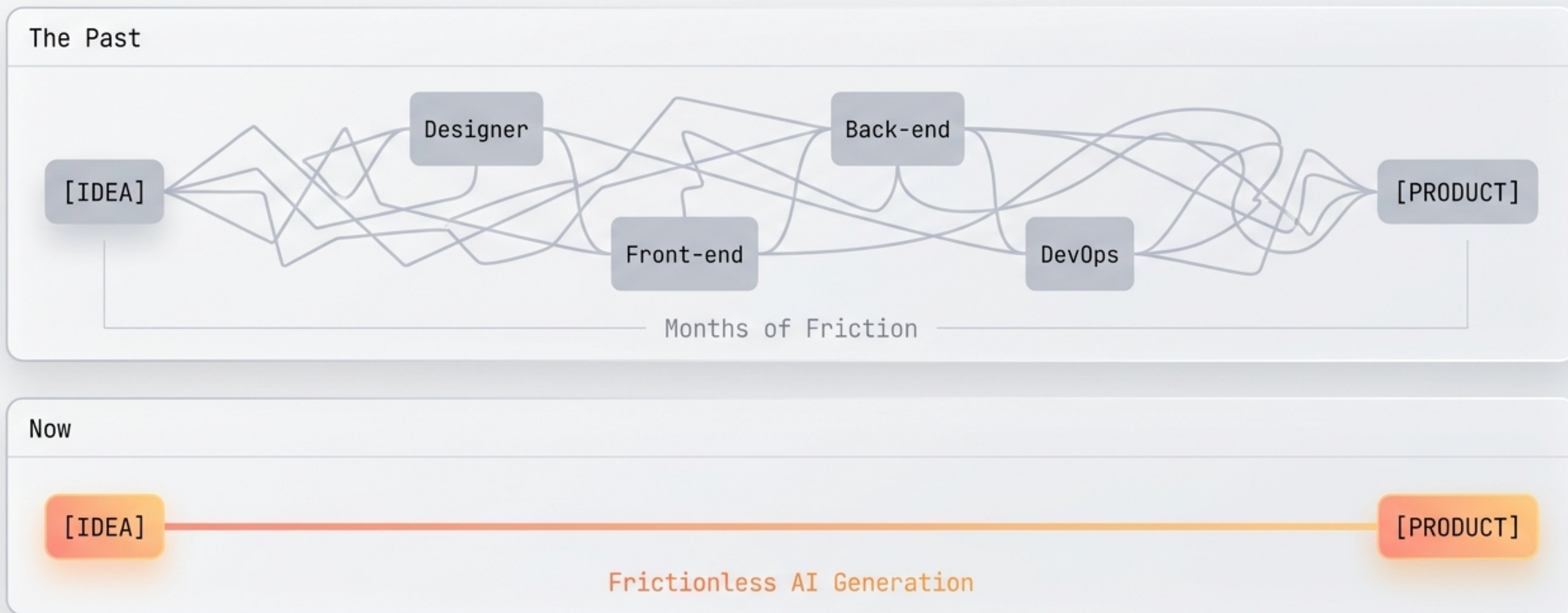


特別講義 Claude Code Desktop で LP・アプリを作ろう

[STATUS: READY]

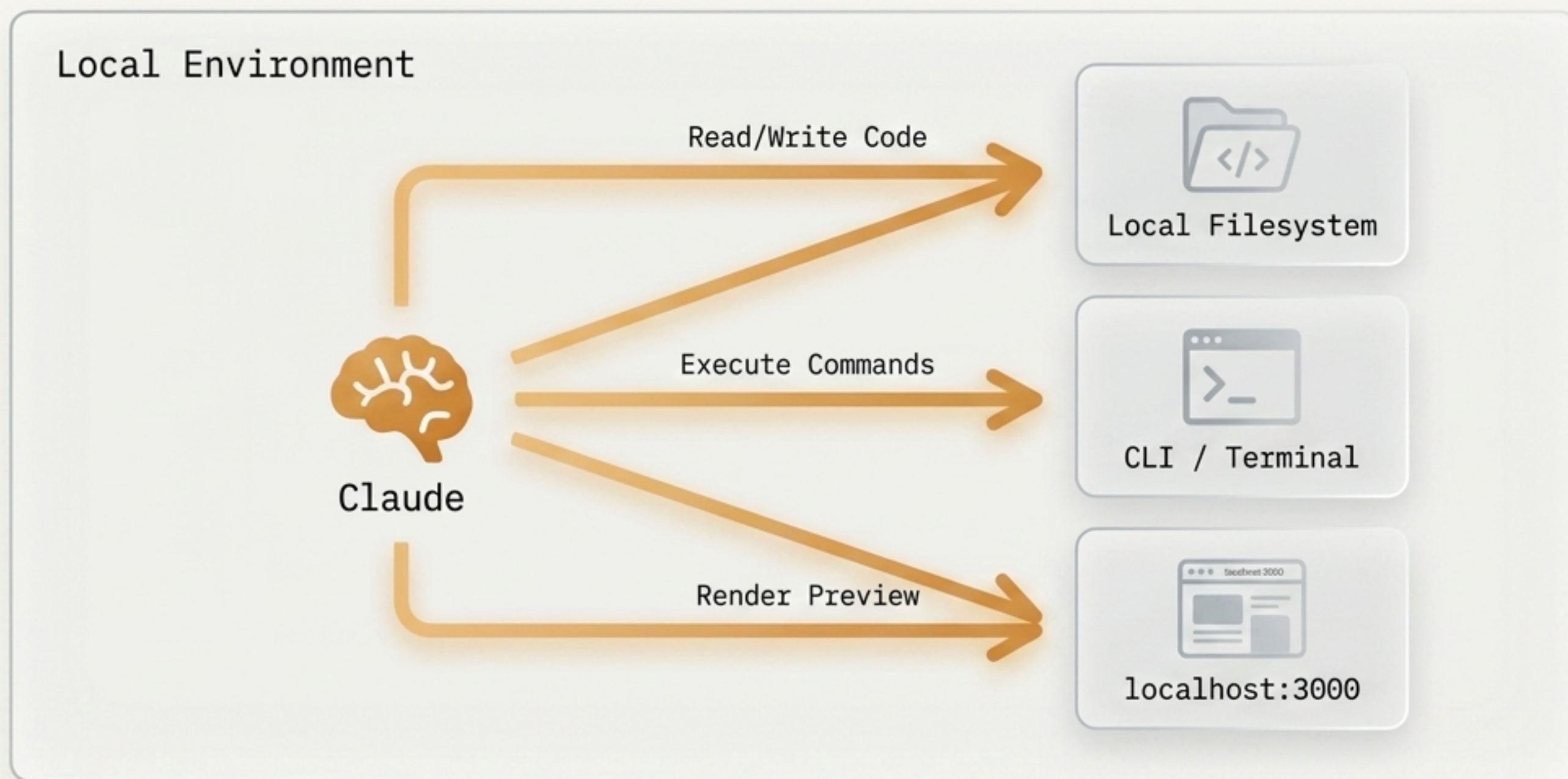
ソフトウェア開発の壁は崩壊した

アイデアを形にするために、もはや大規模なエンジニアリングチームを介す必要はありません。
自然言語による指示が、そのままプロダクションレベルのコードとして実体化する時代への移行。



チャットボットではなく、ローカル環境に住むAIエンジニア

Claude Codeは単なる対話UIではありません。あなたのPC内に常駐し、ファイル構造を理解し、自律的にターミナルコマンドを実行し、サーバーを立ち上げる「自律型開発エージェント」です。

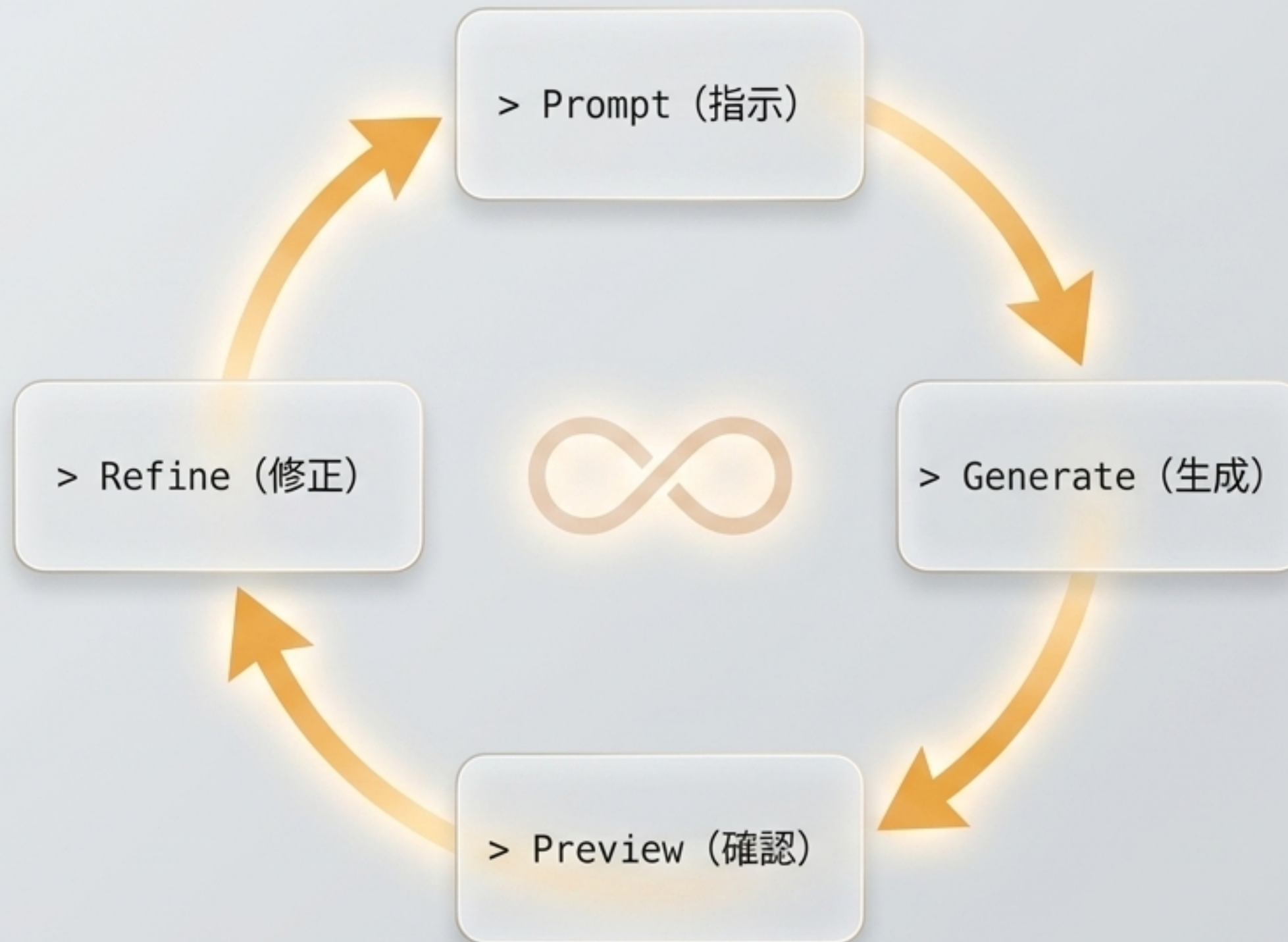


開発パラダイムの移行

	Traditional Dev	Claude Code
プロトタイプ完成までの時間	数週間	数分～数時間
必要ロール	デザイナー・エンジニア複数名	ソロクリエイター1名
デバッグの手間	スタックトレースの手動解読	エラーログを渡すだけの自動修正
イテレーション速度	低い・コスト高	圧倒的高速

開発の新しい基本動作：無限の反復ループ

これからの開発者に求められるスキルは、コードを一行ずつ書くことではありません。この4つのステップからなるループを、いかに高速かつ正確に回し、AIをゴールへと導くかという「ディレクション能力」です。



Phase 1: ランディングページ（LP）の構築

ゼロから美しく、レスポンスな静的ウェブサイトを構築するアプローチ。



プロンプト・ブループリント：デザインを言葉で定義する

曖昧な指示は曖昧なUIを生み出します。「コンテキスト自体がコードである」という前提に立ち、要件を構造化してClaudeに渡す必要があります。

[Target Audience]
ターゲットオーディエンス

```
<prompt_blueprint>
  <context>
    私たちは新しいSaaS製品のランディングページを作成しています。
  </context>
  <audience>
    この製品のターゲットオーディエンスは、最新のテクノロジーに関心があり、効率化を求める中小企業の経営者やチームリーダーです。彼らは専門的な知識は持っていますが、技術的な詳細よりも具体的なメリットやROIを重視します。
  </audience>
  <style>
    ブランドのトナリは「洗練」「信頼」「革新」です。
    カラーパレットは、メインにクリーンなホワイトとライトグレー、アクセントにAnthropicのシグネチャーであるウォームピーチ（#F5B79D）、ソフトアンバー（#FAD6A5）、テラコッタ（#D38E7F）を使用し、活気とモダンさを表現してください。
    タイポグラフィは視認性が高く、モダンなサンセリフ体を選定してください。
  </style>
  <sections>
    このランディングページには以下のセクションが必須です：
    - ヒーローセクション：強力な見出し、サブテキスト、明確なCTAボタン
    - 機能紹介：主要な3つの機能をアイコンと短い説明で紹介
    - 導入メリット：具体的なROIや成功事例を示すセクション
    - 社会的証明：信頼できる顧客の声やロゴ
    - フッター：ナビゲーション、コンタクト情報、SNSリンク
  </sections>
</prompt_blueprint>
```

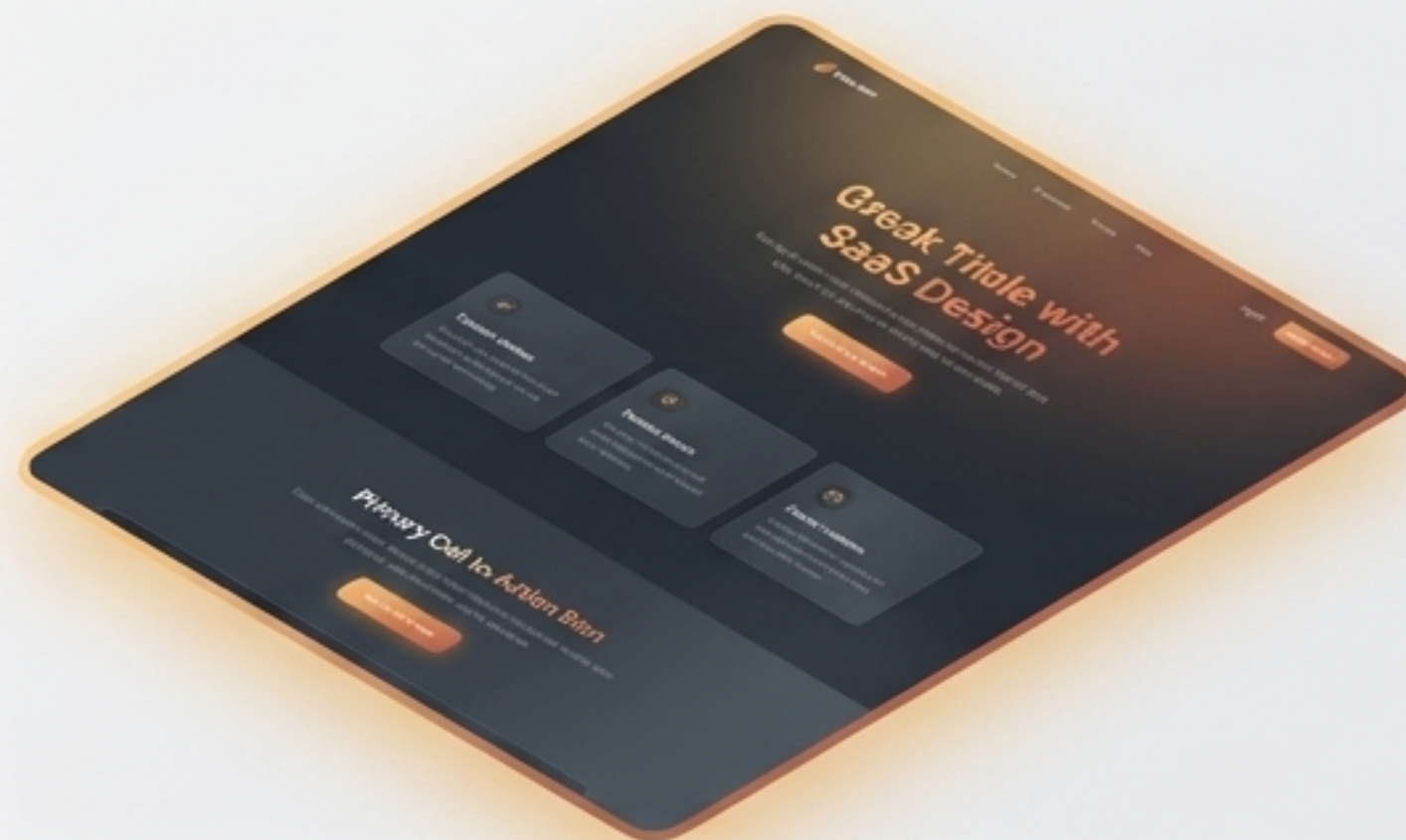
[Brand Colors & Tone]
ブランドカラーとトナリ

[Desired Sections]
必要なセクション構成

意図からUIへの即時変換

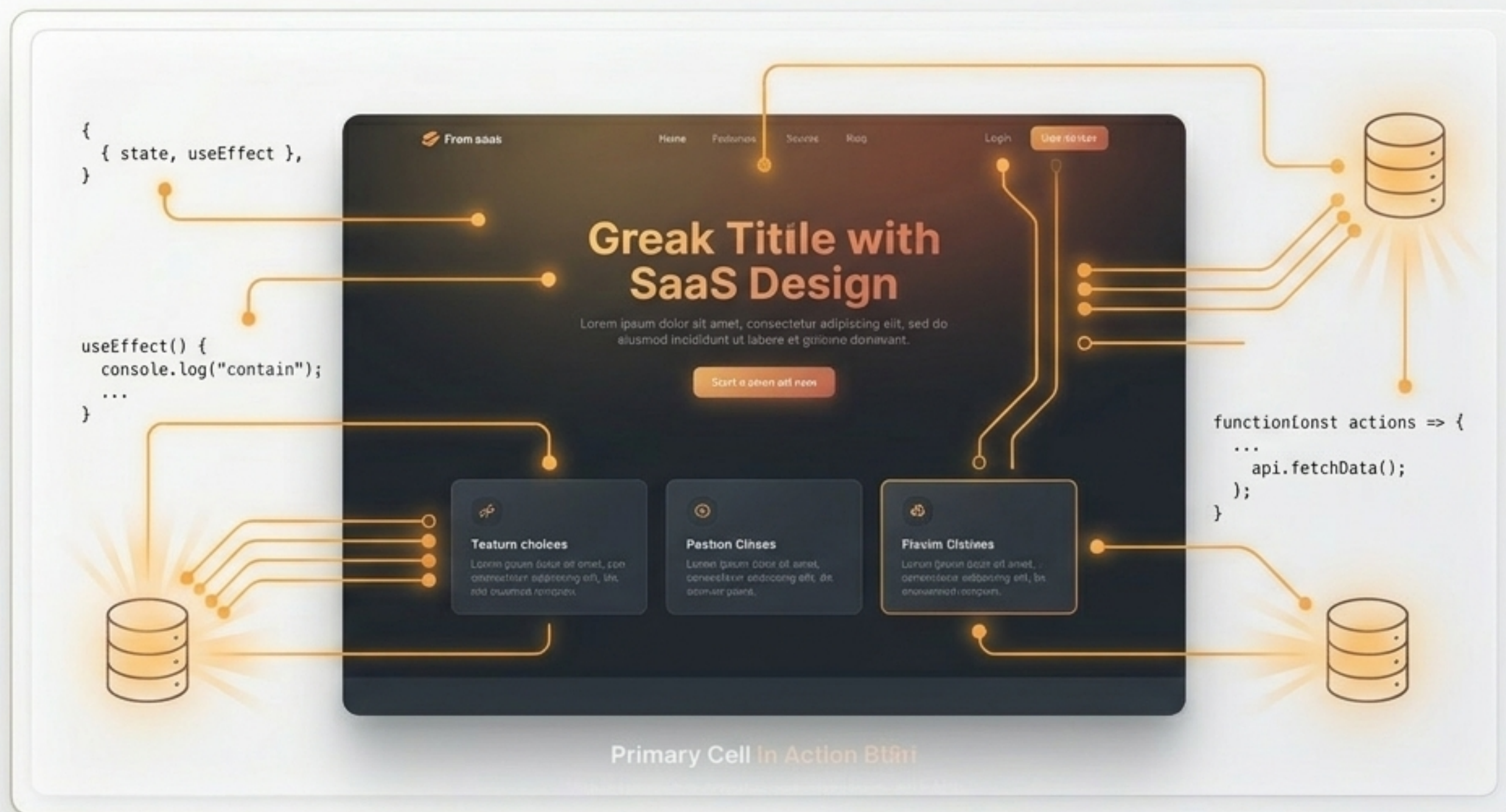
高度な設計意図の言語化だけで、コンポーネントベースのフロントエンドコード（React / Tailwind）が即座に生成され、ローカル環境でプレビュー可能になります。

> ダークテーマのモダンなSaaS向けLPを作成して。ReactとTailwindを使用すること。



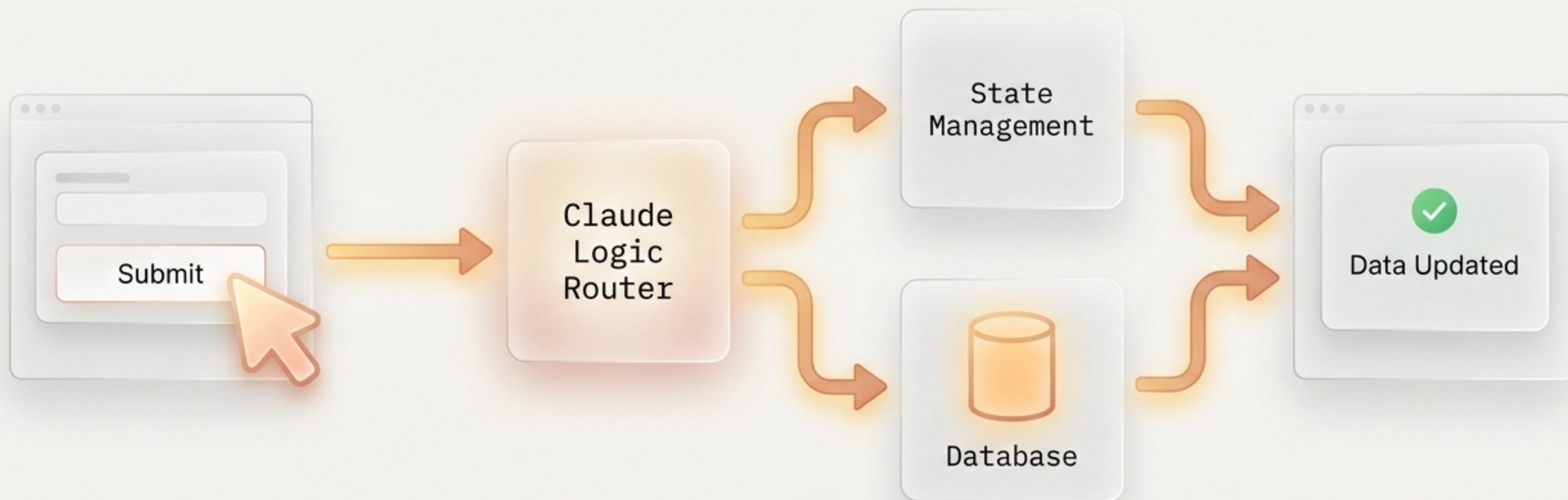
Phase 2: 動的アプリケーションの構築

静的なデザインから、データとロジックを伴う機能的なアプリケーションへ。



状態管理 & 管理とロジックをAIに委ねる

アプリケーション開発では「見た目」ではなく「機能」を指示します。データベースの構造、ユーザー認証のフロー、状態管理（State）のルーティングについて、Claudeと対話しながら要件を定義します。



アプリ・ブループリント：振る舞いとデータ構造の設計

動的アプリの構築において最も重要なのは、正常系だけでなく、データが空の場合やエラー発生時の「振る舞い」を事前に言語化しておくことです。

[Data Structure / Schema]
データ構造の定義

```
// アプリケーション設計プロンプト
[
  {
    "entity": "ユーザー",
    "schema": {
      "id": "string (UUID)",
      "email": "string",
      "preferences": {
        "theme": "string (light/dark)",
        "notifications": "boolean"
      }
    }
  },
  {
    "action": "ログイン",
    "input": { "email": "string", "password": "string" },
    "output": "セッション・トークン"
  }
]
```

```
### ユーザー操作フロー ###
1. ユーザーがログイン画面にアクセスする。
2. メールアドレスとパスワードを入力し、「ログイン」ボタンをクリックする。
3. システムが認証情報を検証する。
4. 成功時: ダッシュボードにリダイレクトされる。
5. 失敗時: エラーメッセージ「メールアドレスまたはパスワードが正しくありません」が表示される。
6. ユーザーが「パスワードをお忘れですか?」をクリックする。
   a. パスワードリセット画面に移動する。
```

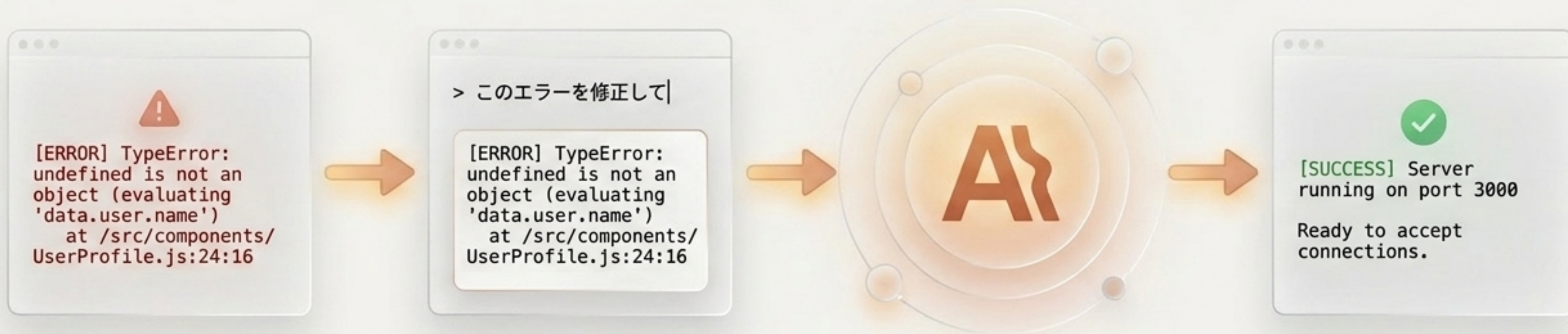
```
### エッジケースとエラーハンドリング ###
- ネットワーク接続が切断された場合:
  - 「インターネット接続を確認してください」というトーストメッセージを表示する。
  - 操作をリトライするボタンを提示する。
- APIサーバーからの応答がタイムアウトした場合:
  - 「サーバーが応答しません。しばらくしてから再試行してください」と表示する。
  - フィードバックフォームへのリンクを提供する。
- データが空の場合 (例: ユーザーの投稿履歴なし):
  - 空の状態のイラストとメッセージ「まだ投稿がありません。最初の投稿を作成しましょう!」を表示する。
  - 新規投稿ボタンを強調表示する。
- 入力バリデーションエラー:
  - 各入力フィールドの下に赤字で具体的なエラーメッセージ (例: 「メールアドレスの形式が正しくありません」) を表示する。
```

[User Interaction Flow]
ユーザーの操作フロー

[Edge Cases]
エッジケースとエラーハンドリング

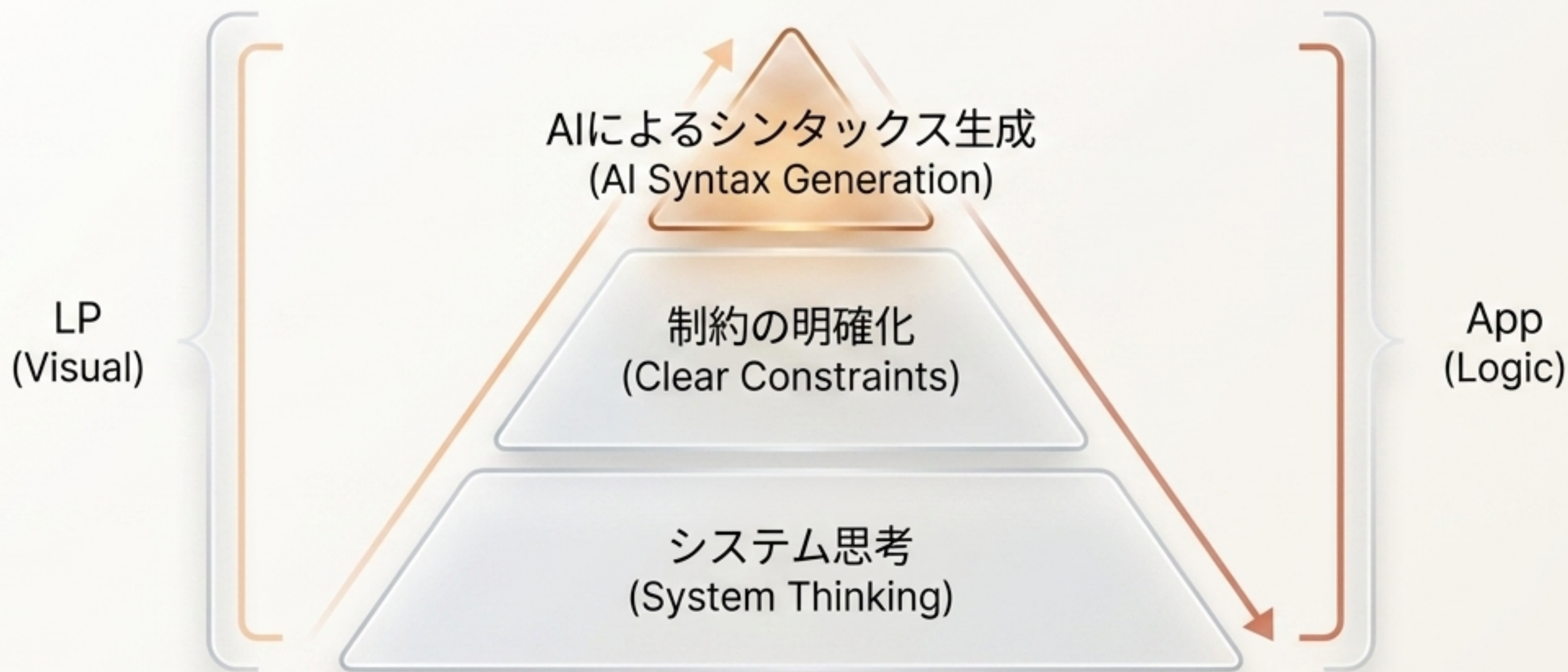
エラーは壁ではなく、対話の一部である

スタックトレースを読み解く必要はありません。ターミナルに出力されたエラーログをそのままClaudeに渡すだけで、問題の特定からコードの修正、再実行までがワンストップで完了します。



統合型AI開発フレームワーク

単純な1ページのLPであれ、複雑な状態管理を持つアプリであれ、根底にある人間とAIのインタラクションモデルは全く同じです。複雑性が増しても、この方法論はスケールします。システムとして思考し、文法はAIに任せましょう。



Claude Codeを操る3つの鉄則

AIは心を読む魔法使いではなく、極めて優秀なジュニアエンジニアとして扱うべきです。
一度に全てを作らせるのではなく、細かく要件を切り出し、対話を通じてシステムを組み上げてください。



制約を明確に
(Be specific with constraints)



小さなステップで反復
(Iterate in small steps)



思考を言語化する
(Verbalize your thoughts)

デジタル・アトリエへの招待

あなたのアイデアを実体化するための全ては、すでにデスクトップ上に揃っています。
今すぐ、最初のコマンドを打ち込みましょう。

```
> start
```